Proyecto 2: Simulador de gestión de memoria paginada usando árboles AVL y Splay Trees.

Nov 3, 2026.

Patrick Zúñiga Arroyo - 2025104944

1. Introducción

Este proyecto es un simulador de consola en C++ que administra la memoria de una computadora dividida en páginas. Varios procesos piden memoria, la usan, piden más y la liberan. El simulador lleva el control de qué páginas físicas están libres, qué páginas tiene cada proceso y qué páginas se usaron hace poco (cachés). Las operaciones se leen desde un archivo de texto o desde la consola, y después de cada una se muestra el estado completo de la memoria.

Alcance. Todo lo que pide el enunciado está implementado y probado: las cinco operaciones, los dos modos de entrada, el caché local y el caché global con política LRU, los nueve tipos de error y las métricas. Como extras se hicieron pruebas automáticas por módulo (203 comprobaciones), la exportación de métricas a CSV y la visualización de los árboles en consola (sección 16).

Compilar y ejecutar (Windows, MinGW g++ 6.3). El compilador usa C++14:

```
g++ -std=c++14 -Wall -Wextra -g main.cpp OperationParser.cpp
MemoryManager.cpp Process.cpp Page.cpp Cache.cpp -o proyecto.exe
```

También se puede usar `compilar.bat` y luego `.\proyecto.exe` (menú) o `.\proyecto.exe entrada_prueba.txt` (procesa el archivo directamente).

2. Descripción del problema

Un sistema operativo divide la memoria física en bloques del mismo tamaño llamados **páginas**. Cada proceso ve su propio espacio de direcciones (direcciones virtuales), dividido en páginas virtuales, y una **tabla de páginas** dice en qué página física está cada una. Al referenciar una dirección se calcula:

```
paginaVirtual  = direccion / TAMANO_PAGINA
desplazamiento = direccion % TAMANO_PAGINA
```

Con los parámetros del enunciado (MEMORIA_TOTAL 65536 y TAMANO_PAGINA 1024) hay 65536 / 1024 = 64 páginas físicas. Por ejemplo, `ACCESS P1 2500` cae en la página virtual 2 con desplazamiento 452 (2500 = 2 × 1024 + 452).

Como consultar la tabla de páginas en cada acceso es costoso, el simulador usa dos **cachés**: uno local por proceso y uno global compartido. Si la página está en un caché se cuenta un *hit*; si no, un *miss*. El enunciado pide administrar las páginas físicas libres con un árbol AVL y la tabla de páginas de cada proceso con un Splay Tree, sin usar `std::map` ni `std::set` en su lugar.

3. Diseño de clases

Cada módulo tiene un .h (qué ofrece) y un .cpp (cómo lo hace). Las plantillas (AVLTree y SplayTree) viven solo en el .h. Están las siete clases que exige el enunciado:

Archivo	Funcionalidad
AVLTree.h	Árbol AVL genérico (plantilla): insertar, eliminar, buscar, rotarIzquierda, rotarDerecha, balancear, inOrden, altura y extraerMinimo. Administra las páginas físicas libres.
SplayTree.h	Splay Tree genérico (plantilla): insertar, eliminar, buscar, splay, rotacionZig, rotacionZigZig, rotacionZigZag y recorrido. Es la tabla de páginas de cada proceso.
Cache	Caché LRU con std::list y std::unordered_map. La misma clase sirve para el caché local y el global.
Page	Una página: página virtual, página física, proceso, bit válida, bit modificada, contador de referencias y última referencia.
Process	Un proceso: id, memoria solicitada, tabla de páginas (Splay Tree) y caché local.
MemoryManager	El simulador: AVL de páginas libres, tabla de procesos, caché global, métricas, las cinco operaciones y los extras (dibujo de árboles y CSV).
OperationParser	Lee líneas de un archivo o de la consola, revisa el formato y llama al MemoryManager.
main	Menú principal (archivo, consola, salir) o ejecución directa con un archivo.

La regla es que las estructuras y el MemoryManager no leen del teclado ni muestran errores por su cuenta: devuelven `true` o `false` con un mensaje, y el OperationParser decide qué imprimir. Así se relacionan las clases:

```
main
 +-- OperationParser
      +-- MemoryManager
           |-- AVLTree<int, Page>          paginas fisicas libres
           |-- Cache                       cache global
           +-- unordered_map<string, unique_ptr<Process>>
                +-- Process
                     |-- SplayTree<int, Page>   tabla de paginas
                     +-- Cache                  cache local
```

4. Estructuras de datos utilizadas

Árbol AVL (`AVLTree<int, Page>`). Guarda las páginas físicas libres; la clave es el número de página física. Se justifica en la sección 5.

Splay Tree (`SplayTree<int, Page>`). Uno por proceso; la clave es el número de página virtual y el valor es la Page. Se justifica en la sección 6.

Lista doblemente enlazada (`std::list<EntradaCache>`). Guarda el orden de uso de un caché: el frente es la entrada más reciente y el final la menos reciente, que es la que LRU expulsa. Mover una entrada al frente con `splice` no copia nada y cuesta O(1).

Tabla hash (`std::unordered_map`). Tiene dos usos. En Cache encuentra una entrada en O(1) por la clave "idProceso#paginaVirtual" y guarda un iterador a su lugar en la lista. En MemoryManager es la tabla global de procesos (id → Process).

Vector (`std::vector<std::string>`). Guarda el orden de creación de los procesos, para mostrarlos siempre en el mismo orden (la tabla hash no tiene orden).

Puntero inteligente (`std::unique_ptr<Process>`). Process no se puede copiar porque tiene un árbol y un caché, así que la tabla de procesos guarda punteros que borran el proceso solos en END_PROCESS.

Ninguna de estas estructuras reemplaza al AVL ni al Splay Tree: el enunciado permite vector, list y unordered_map como auxiliares.

5. Justificación del árbol AVL

El AVL administra las páginas físicas libres. Cada nodo guarda su altura y, después de insertar o eliminar, se compara la altura de sus dos lados (factor de balance). Si la diferencia es mayor que 1 se hace una rotación simple o doble que lo reacomoda. Así el árbol nunca degenera en una lista.

Por qué AVL aquí. Al configurar la memoria, las 64 páginas se insertan en orden (0, 1, 2, …). Un árbol de búsqueda sin balance quedaría como una línea de altura 64 y cada operación costaría O(n). El AVL queda con altura 7 y garantiza O(log n) en el peor caso, no solo en promedio. Eso importa porque el AVL se usa en casi todas las operaciones: NEW_PROCESS y ALLOCATE sacan páginas, y FREE y END_PROCESS las devuelven.

Qué página se asigna. Siempre la libre de número más bajo, con `extraerMinimo()`: baja a la izquierda hasta el final y elimina ese nodo. Así la asignación es predecible y fácil de verificar: P1 recibe las páginas 0, 1, 2 y 3, el siguiente proceso recibe de la 4 en adelante, y una página liberada es la primera en volver a usarse.

Rotaciones. Se implementaron los cuatro casos: izquierda-izquierda (`rotarDerecha`), derecha-derecha (`rotarIzquierda`), izquierda-derecha (`rotarIzquierda` sobre el hijo y luego `rotarDerecha`) y derecha-izquierda (al revés). La función `balancear()` se llama al volver de cada llamada recursiva de insertar y eliminar, y `estaBalanceado()` verifica en las pruebas que todo el árbol cumpla la regla.

El AVL se reutilizó del árbol balanceado (ABB.h) del Proyecto 1, que ya estaba probado, y se adaptó a los nombres del enunciado (`inOrden`) y a esta necesidad (`extraerMinimo`).

6. Justificación del Splay Tree

Cada proceso tiene un Splay Tree como tabla de páginas. Es un árbol de búsqueda que no guarda alturas: cada vez que se consulta, inserta o elimina una clave, la sube a la raíz con rotaciones (*splay*). Las páginas que se usan seguido quedan cerca de la raíz y se encuentran en pocos pasos.

Por qué Splay aquí. Los programas reales tienen **localidad de referencia**: cuando usan una página, es muy probable que la vuelvan a usar pronto (un ciclo, un arreglo). El Splay Tree aprovecha esto: un acceso repetido a la misma página la encuentra en la raíz, en O(1). Su costo amortizado es O(log n) por operación, aunque una operación aislada pueda costar O(n).

Las tres rotaciones. Para subir un nodo x con padre p y abuelo g:

- **Zig** (`rotacionZig`): x es hijo de la raíz. Basta una rotación simple.
- **Zig-zig** (`rotacionZigZig`): x y p están del mismo lado. Se rota primero g y luego p.
- **Zig-zag** (`rotacionZigZag`): x está del lado contrario a p. Se rota primero p hacia el otro lado y luego g.

Splay en todos los accesos. El enunciado pide hacer splay sobre la página consultada. Por eso ACCESS busca la página en el Splay Tree siempre, aunque haya sido hit en un caché; así el árbol refleja el patrón real de uso. Para eliminar, se sube la clave a la raíz, se quita y se unen los dos lados subiendo el mayor del lado izquierdo.

AVL frente a Splay. El AVL garantiza el peor caso, lo que conviene para las páginas libres, que se usan de forma pareja. El Splay Tree se adapta al uso, lo que conviene para la tabla de páginas, donde unas pocas páginas concentran la mayoría de los accesos.

7. Integración entre módulos

Las estructuras no trabajan aisladas. Un solo ACCESS las usa casi todas:

1. **Lectura.** OperationParser lee `ACCESS P1 2500`, revisa que tenga dos datos y que 2500 sea un número.
2. **Proceso.** MemoryManager busca P1 en la tabla hash de procesos.
3. **Dirección.** Calcula la página virtual 2 y el desplazamiento 452, y revisa que la página esté dentro del espacio de P1.
4. **Caché local.** Busca P1:2 en el caché de P1. Si está, es hit local.
5. **Caché global.** Si no estaba, busca P1:2 en el caché global. Si está, es hit global.
6. **Splay Tree.** Busca la página 2 en la tabla de P1 y la sube a la raíz. De ahí sale la página física.
7. **Métricas y LRU.** Suma hits o misses, actualiza el contador de la página y la mete en los cachés; si alguno está lleno, LRU expulsa la entrada menos reciente.

FREE y END_PROCESS recorren el camino contrario: sacan las páginas del Splay Tree, las devuelven al AVL de libres y las quitan de los dos cachés, para que ningún caché apunte a una página que ya no existe.

8. Decisiones de diseño

El enunciado deja algunas decisiones al estudiante. Estas fueron las tomadas:

Decisión	Qué se hizo y por qué
Tamaño del caché local	max(1, floor(0.20 × páginas)). En el código es páginas / 5 con división entera, que da lo mismo sin usar decimales. Con 4 páginas da 1 y con 10 da 2. Se recalcula en NEW_PROCESS, ALLOCATE y FREE; si baja, LRU expulsa las entradas que sobran.
Página física asignada	La libre de número más bajo (extraerMinimo del AVL).
Numeración virtual	0, 1, 2, … en orden de asignación. ALLOCATE continúa desde la última, aunque FREE haya dejado huecos.
Dirección inválida o página no válida	Una dirección negativa o fuera del espacio dado al proceso es "Dirección inválida". Una dentro del espacio, pero liberada con FREE, es "Acceso a página no válida".
FREE es todo o nada	Si alguna página del rango no está asignada, no se libera ninguna y se informa cuál falla.
Orden de búsqueda en ACCESS	Caché local, luego global, luego Splay Tree, como dice el enunciado. Los contadores del global solo cambian cuando hubo miss local, así que hits + misses globales = misses locales.
Splay en cada acceso	Siempre, también cuando hay hit en un caché.
Operación con error	No cambia la memoria y no cuenta en las métricas.
Configuración	Los tres parámetros pueden venir en cualquier orden, pero antes de las operaciones. Si es inválida, la simulación se detiene: sin ella no hay memoria que simular.
Límites	Máximo 1 048 576 páginas y caché global de hasta 1 048 576 entradas, para que un archivo con valores enormes no congele el programa.
Dibujo de árboles	ARBOL no hace splay ni cambia nada: solo lee el árbol y lo dibuja, así que mostrarlo no altera las métricas ni la forma del Splay Tree.

Decisión	Qué se hizo y por qué
Formato CSV	Dos tablas en el mismo archivo: métricas generales (metrica,valor) y, después de una línea vacía, una fila por proceso activo. Los números no llevan separador de miles y el decimal es un punto.

9. Formato de entrada

Una operación por línea, con los datos separados por espacios. Las tres líneas de configuración van antes de las operaciones, en cualquier orden. Lo que está después de # es comentario y las líneas vacías se ignoran. Los comandos aceptan mayúsculas o minúsculas; los id de proceso no.

Línea	Qué hace
`MEMORIA_TOTAL n`	Bytes de memoria física.
`TAMANO_PAGINA n`	Bytes por página. MEMORIA_TOTAL debe ser múltiplo de este valor.
`TAMANO_CACHE_GLOBAL n`	Cantidad de entradas del caché global.
`NEW_PROCESS id bytes`	Crea un proceso con bytes / TAMANO_PAGINA páginas, redondeando hacia arriba.
`END_PROCESS id`	Termina el proceso y libera todo lo suyo.
`ACCESS id direccion`	Referencia una dirección virtual del proceso.
`ALLOCATE id bytes`	Le da páginas adicionales al proceso.
`FREE id direccionInicial bytes`	Libera las páginas virtuales del rango.
`ESTADO`, `METRICAS`, `AYUDA`, `SALIR`	Muestran el estado, las métricas o la ayuda, o terminan la lectura.
`ARBOL AVL`	Extra: dibuja el AVL de páginas físicas libres.
`ARBOL SPLAY idProceso`	Extra: dibuja la tabla de páginas (Splay Tree) del proceso.
`EXPORTAR archivo.csv`	Extra: guarda las métricas y la tabla de procesos en un archivo CSV.

Ejemplo (`entrada_prueba.txt`, el mismo del enunciado):

```
MEMORIA_TOTAL 65536
TAMANO_PAGINA 1024
TAMANO_CACHE_GLOBAL 8
NEW_PROCESS P1 4096
NEW_PROCESS P2 8192
```

```
ACCESS P1 100
ACCESS P1 2500
ACCESS P2 7000
ALLOCATE P1 2048
FREE P2 2048 2048
END_PROCESS P1
END_PROCESS P2
```

10. Formato de salida

Cada línea procesada se muestra con su número y su resultado y, si salió bien, el estado completo de la memoria. Por ejemplo, la línea 7 del archivo anterior:

```
[Linea 7] ACCESS P1 2500
  Pagina virtual: 2
  Desplazamiento: 452
  Resultado: MISS en cache local, MISS en cache global (se obtuvo del Splay
Tree)
  Cache local lleno: LRU expulsa P1:0
  Pagina fisica: 2
  Estado de la memoria:
  Paginas fisicas: 64 | libres: 52 | ocupadas: 12
  Paginas libres (AVL en orden): 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52
53 54 55 56 57 58 59 60 61 62 63
  Procesos activos:
    P1 | memoria solicitada: 4096 bytes | paginas: 4 | cache local (1/1):
[P1:2->2]
    P2 | memoria solicitada: 8192 bytes | paginas: 8 | cache local (0/1): []
  Cache global (2/8): [P1:2->2 P1:0->0]
  Hits/misses -> local: 0/2 | global: 0/2
```

Cómo leerla. `P1:2->2` significa proceso P1, página virtual 2, página física 2. En los cachés la primera entrada es la más reciente. `(1/1)` es entradas usadas / capacidad. `Hits/misses` son los contadores acumulados. Los errores se muestran como `ERROR: <tipo>: <detalle>`, seguido de `(La memoria no cambio.)`.

Al final se imprime un **resumen**: operaciones exitosas, errores, las métricas obligatorias (total de accesos, hits y misses local y global, páginas asignadas y libres, porcentaje de utilización) y el estado final.

11. Manejo de errores

Cada mensaje empieza con el nombre del error del enunciado, para identificarlo fácil. Ningún error detiene el programa, salvo los de configuración.

Error	Cuándo ocurre	Ejemplo
Proceso inexistente	El id no está en la tabla de procesos (ACCESS, ALLOCATE, FREE, END_PROCESS).	`ACCESS P9 0`

Error	Cuándo ocurre	Ejemplo
Proceso duplicado	NEW_PROCESS con un id que ya existe.	`NEW_PROCESS P1 1024`
Memoria insuficiente	No hay suficientes páginas libres en el AVL.	`NEW_PROCESS P4 60000`
Dirección inválida	La dirección es negativa o está fuera del espacio del proceso.	`ACCESS P3 5000`
Liberación de memoria no asignada	FREE con alguna página del rango sin asignar, o con 0 bytes.	`FREE P1 2048 1024` (ya liberada)
Acceso a página no válida	La página está dentro del espacio del proceso pero se liberó con FREE.	`ACCESS P1 2500` después de liberarla
Archivo de entrada incorrecto	No se puede abrir el archivo, falta o se repite la configuración, faltan datos o un número está mal escrito.	`ACCESS P1 abc`
Tamaño de página inválido	TAMANO_PAGINA es 0 o negativo, es mayor que la memoria, o la memoria no es múltiplo de él.	`TAMANO_PAGINA 1000`
Operación desconocida	El comando no existe.	`MOVE P1 0`

En modo consola, si la configuración es inválida se vuelve a pedir, y si se escribe una letra donde va un número, se pide otra vez.

12. Análisis de complejidad

N = páginas físicas, p = páginas de un proceso, k = páginas que toca una operación (bytes / TAMANO_PAGINA), c = tamaño de un caché, C = entradas en todos los cachés, P = procesos activos. El AVL se balancea solo, así que su altura es siempre O(log N). El Splay Tree es amortizado: una operación aislada puede costar O(p), pero una secuencia de m operaciones cuesta O(m log p).

Operación	Estructura	Promedio	Peor caso
Configurar la memoria	AVL	O(N log N)	O(N log N)
Buscar, insertar o eliminar una página libre	AVL	O(log N)	O(log N)
Tomar la página libre más baja	AVL	O(log N)	O(log N)

Operación	Estructura	Promedio	Peor caso
Buscar, insertar o eliminar una página virtual (con splay)	Splay Tree	O(log p) amortizado	O(p)
Buscar un proceso	unordered_map	O(1)	O(P)
Buscar, insertar o expulsar en un caché (LRU)	list + unordered_map	O(1)	O(c)
Quitar las entradas de un proceso del caché global	list	O(c)	O(c)
NEW_PROCESS / ALLOCATE	AVL + Splay	O(k (log N + log p))	O(k (log N + p))
ACCESS	Cachés + Splay	O(log p) amortizado	O(p)
FREE	Splay + AVL + cachés	O(k (log p + log N))	O(k (p + log N))
END_PROCESS	Splay + AVL + cachés	O(p log N + c + P)	O(p log N + c + P)
Mostrar el estado	Todas	O(N + P + C)	O(N + P + C)

Espacio. Cada página física está en el AVL (libre) o en el Splay Tree de un proceso (asignada), nunca en los dos, así que los árboles ocupan O(N) en total. Los cachés agregan O(C).

13. Validaciones, memoria y buenas prácticas

Validaciones de entrada: Los números se leen como texto y se convierten a mano: solo se aceptan dígitos, con signo opcional y hasta 15 cifras, así que `12abc`, `3.5` o un número gigante se rechazan sin romper el programa. Se ignoran los comentarios, el `\r` de los archivos de Windows y los bytes invisibles (BOM) que agrega el Bloc de notas.

Memoria: Los nodos de los árboles se crean con `new` y se liberan en postorden en los destructores. Las clases que manejan memoria (AVLTree, SplayTree, Cache, Process, MemoryManager y OperationParser) tienen el constructor de copia borrado. Los procesos se guardan en `std::unique_ptr`. Las pruebas y el programa se revisaron con AddressSanitizer: cero fugas y cero accesos inválidos.

Buenas prácticas:

- Un módulo por responsabilidad, con .h y .cpp separados.
- `const` en los métodos que solo consultan.
- Sin `std::to_string` ni nada fuera de C++14, para compilar con MinGW 6.3.
- Las operaciones devuelven `bool` y un mensaje; no imprimen errores por su cuenta.
- Una operación que falla no deja la memoria a medias: FREE revisa todo el rango antes de liberar.
- Comentarios cortos que explican el porqué.

14. Dificultades encontradas

- **Splay recursivo.** Hubo que separar bien los tres casos y revisar que el nieto exista antes de una rotación doble; si no existe, basta una rotación simple. Se probó con 101 claves en orden revuelto y eliminando la mitad.
- **Eliminar en el Splay Tree.** Al borrar la raíz quedan dos subárboles sueltos. Se resolvió subiendo el mayor del lado izquierdo, que queda sin hijo derecho, y colgándole el lado derecho.
- **LRU en O(1).** Con una lista sola hay que recorrerla para buscar. Se combinó con una tabla hash que guarda iteradores; `splice` mueve el nodo al frente sin invalidar el iterador.
- **Process no se puede copiar.** Por tener un árbol y un caché, no se podía guardar por valor en el `unordered_map`. Se usó `unique_ptr`.
- **Dirección inválida o página no válida.** Con los huecos que deja FREE ya no basta comparar contra la cantidad de páginas. Se agregó `siguientePaginaVirtual` para distinguir los dos errores.
- **Archivos de Windows.** El `\r` al final de cada línea y el BOM del Bloc de notas hacían que los comandos no se reconocieran. Se limpian al leer.
- **Compilador antiguo.** MinGW 6.3 no tiene `std::to_string`; se usó `ostringstream`.
- **Dibujar el árbol.** Había que decidir cómo mostrar un árbol en texto. Se imprime de lado: primero el subárbol derecho, después el nodo con sangría según su nivel y al final el subárbol izquierdo. La sangría muestra el nivel sin caracteres de dibujo, así que se ve igual en cualquier consola.

15. Limitaciones conocidas y mejoras futuras

- No se simulan escrituras: `bitModificada` existe en Page, pero ninguna operación del enunciado escribe en memoria. Mejora: una operación WRITE.
- No hay memoria secundaria: si no hay páginas libres, NEW_PROCESS y ALLOCATE fallan. Mejora: reemplazo de páginas físicas a disco (swap) con LRU.
- Un id de proceso no puede contener #, porque se usa para comentarios y en la clave del caché.
- La lista de páginas libres se muestra completa en cada estado; con memorias grandes la salida es larga. Mejora: mostrarla por rangos (0-15, 20-63).
- La interfaz es solo de consola. Mejora: interfaz gráfica.

16. Extras implementados

El enunciado ofrece puntos extra por exportar las métricas y por visualizar los árboles. Ambos están implementados y probados.

16.1 Exportación de métricas a CSV

El comando `EXPORTAR archivo.csv` escribe las métricas en un archivo CSV: valores separados por comas, que se puede abrir con Excel o LibreOffice. El archivo tiene dos tablas. La primera son las métricas generales (accesos, hits y misses locales y globales, páginas asignadas y libres, utilización). La segunda, después de una línea vacía, tiene una fila por cada proceso activo.

```
EXPORTAR metricas.csv
```

```
PS C:\...\Proyecto2> Get-Content metricas.csv
metrica,valor
memoria_total,8192
tamano_pagina,1024
paginas_fisicas,8
accesos_totales,1
hits_cache_local,0
misses_cache_local,1
hits_cache_global,0
misses_cache_global,1
paginas_asignadas,4
paginas_libres,4
utilizacion_porcentaje,50.00

proceso,memoria_solicitada,paginas_asignadas,cache_local_usada,cache_local_capac
idad
P1,3072,3,1,1
P2,1024,1,0,1
```

Si Excel muestra todo en una sola columna, es porque su configuración regional usa punto y coma como separador. En ese caso se abre con **Datos → Desde texto/CSV** y se elige la coma como separador.

16.2 Visualización de árboles en consola

Los comandos `ARBOL AVL` y `ARBOL SPLAY idProceso` dibujan el árbol en la consola. Se dibujan de lado: la raíz es la línea sin sangría y cada nivel más profundo se corre seis espacios a la derecha. Los hijos de la derecha van arriba y los de la izquierda abajo, así que al leer de arriba hacia abajo las claves salen de mayor a menor.

- **ARBOL AVL** muestra las páginas físicas libres. Sirve para ver que el árbol sigue balanceado.
- **ARBOL SPLAY idProceso** muestra la tabla de páginas de un proceso. La raíz es la última página consultada, porque cada ACCESS la sube a la raíz. Dibujar el árbol no hace splay, así que no cambia su forma.
- Si el proceso no existe, da el error `Proceso inexistente`. Si no se escribe el tipo, da error de formato.

```
ARBOL AVL
ARBOL SPLAY P1
```

```
PS C:\...\Proyecto2> .\proyecto.exe entrada_arbol.txt
...
[Linea 7] ARBOL AVL
  AVL de paginas fisicas libres (clave = numero de pagina).
  La raiz es la linea sin sangria; los hijos de la derecha van arriba
  y los de la izquierda abajo.
           7
      6
5
      4

------------------------------------------------------------
[Linea 8] ARBOL SPLAY P1
  Splay Tree de P1 (clave = pagina virtual).
  La raiz es la linea sin sangria; suele ser la pagina usada mas recientemente.
      2
1
      0

------------------------------------------------------------
[Linea 9] ARBOL SPLAY P9
  ERROR: Proceso inexistente: P9
```

16.3 Otras facilidades

- Pruebas automáticas por módulo: cinco programas con su propio main y su .bat, 203 comprobaciones (sección 18).
- `ejecutar_pruebas.bat`: genera los cuatro archivos de evidencia `salida_*.txt` y `metricas.csv` en un solo paso.
- Comandos ESTADO, METRICAS, AYUDA y SALIR, en archivo y en consola.
- Comentarios con # y líneas vacías en los archivos de entrada.
- Ejecución directa con un archivo (`proyecto.exe entrada.txt`), útil para guardar la salida con `> salida.txt`.
- Cada expulsión LRU se informa: qué entrada sale y de qué caché.
- Tolerancia a archivos guardados en Windows (`\r` y BOM).

17. Manual de usuario breve

Al ejecutar `proyecto.exe` aparece el menú principal. Se elige con el número y ENTER.

Opción	Para qué sirve
1. Leer operaciones desde un archivo	Pide el nombre del archivo (ENTER usa entrada_prueba.txt), lo procesa completo y muestra el resumen.
2. Escribir operaciones en la consola	Pide MEMORIA_TOTAL, TAMANO_PAGINA y TAMANO_CACHE_GLOBAL, y luego lee operaciones línea por línea hasta SALIR.
3. Salir	Termina el programa.

Archivos .bat:

Archivo	Para qué sirve
compilar.bat	Compila el programa y crea proyecto.exe.
ejecutar_pruebas.bat	Compila y ejecuta los cuatro archivos de entrada; guarda salida_prueba.txt, salida_casos.txt, salida_error_config.txt y salida_extras.txt, y genera metricas.csv.
probar_avl.bat, probar_splay.bat, probar_cache.bat, probar_memoria.bat, probar_parser.bat	Compilan y ejecutan las pruebas de cada módulo.

Flujo recomendado para probar:

1. Doble clic en `compilar.bat`.
2. Ejecutar `proyecto.exe`, opción 1 y ENTER: corre el ejemplo del enunciado.
3. Opción 1 con `entrada_casos.txt`: recorre los 12 casos mínimos y los errores. Cada línea del archivo tiene un comentario con lo que se espera.
4. Opción 2: escribir 65536, 1024 y 4; luego `NEW_PROCESS P1 4096`, dos veces `ACCESS P1 2500` (la segunda es HIT local), `ESTADO` y `SALIR`.
5. Procesar `entrada_extras.txt` (opción 1): muestra los árboles con `ARBOL` y genera `metricas.csv` con `EXPORTAR`.
6. `ejecutar_pruebas.bat` para generar las evidencias.

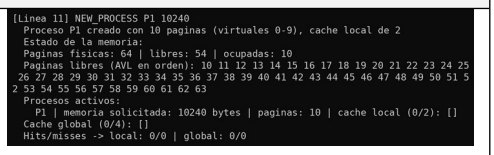
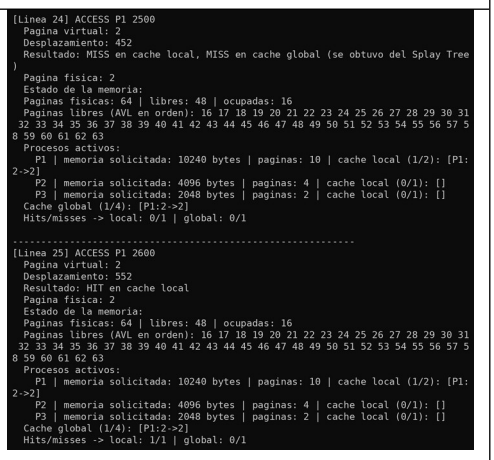
18. Casos de prueba

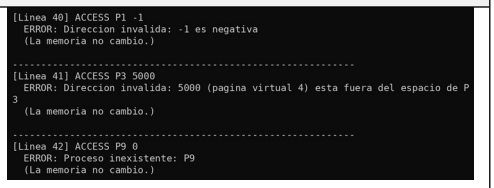
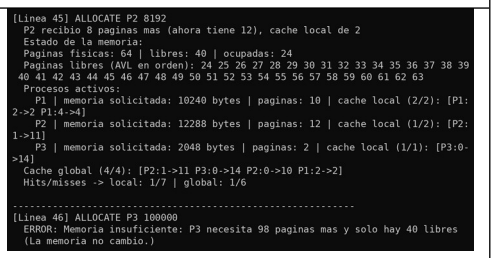
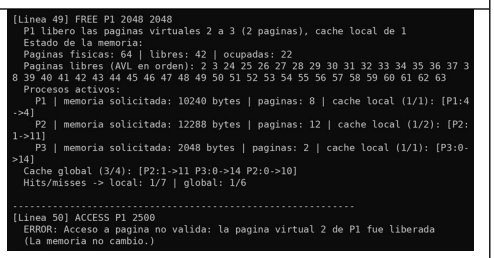
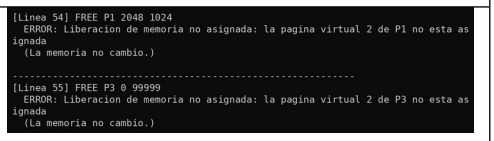
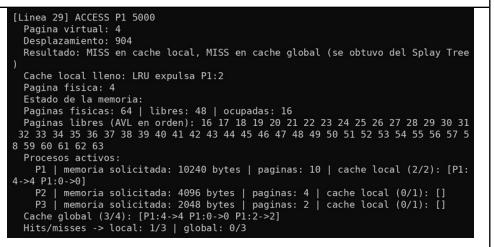
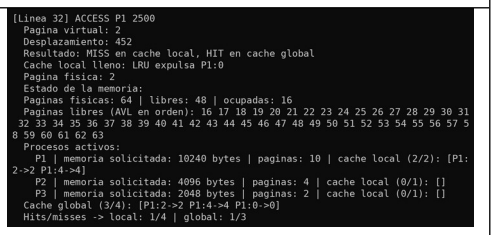
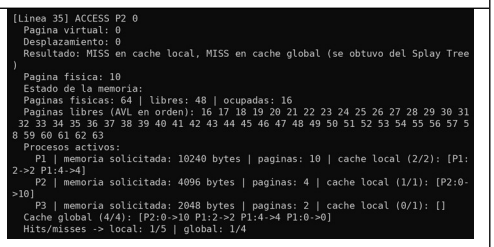
Pruebas automáticas. Cada módulo tiene su programa de prueba con su propio main y un .bat para correrlo (por ejemplo probar_avl.bat). Total: 203 comprobaciones, todas pasan.

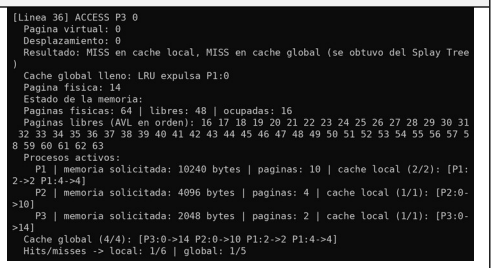
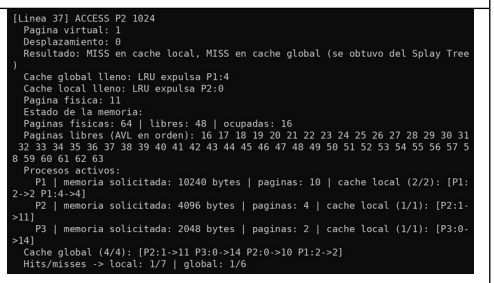
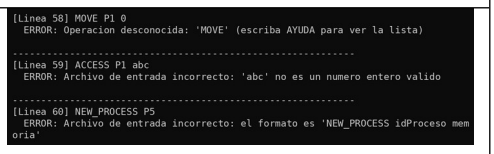
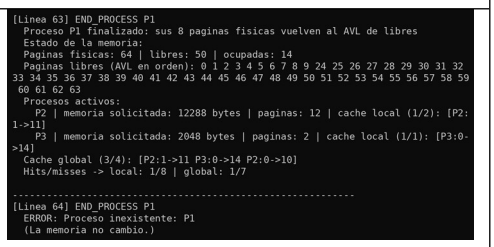
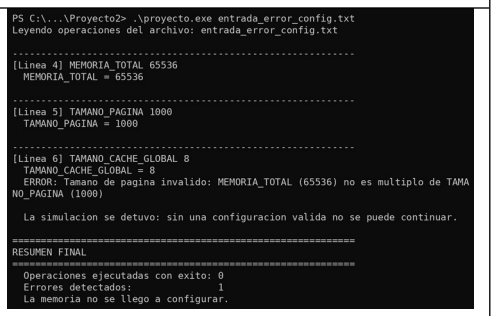
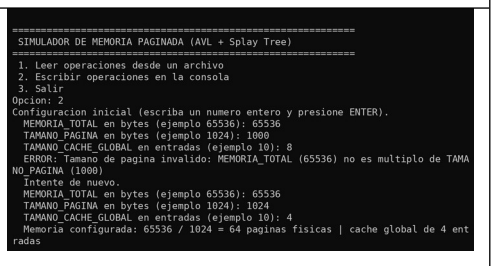
Programa	Qué verifica	Comprobaciones
pruebaAVL	Árbol vacío, insertar, duplicados, buscar, inOrden, 100 claves en orden (balanceo), eliminar (3 casos), extraerMinimo	29
pruebaSplay	Insertar, buscar sube a la raíz, zig-zig, clave inexistente, eliminar, 101 claves revueltas, dibujo del árbol	36
pruebaCache	Expulsión LRU, un hit mueve al frente, duplicados, eliminar, eliminar por proceso, reducir tamaño, tamaño 0	28

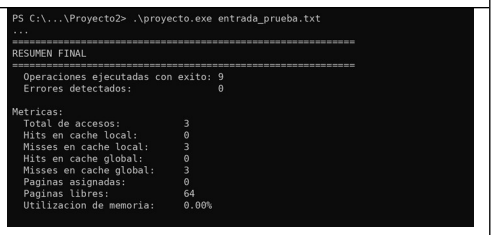
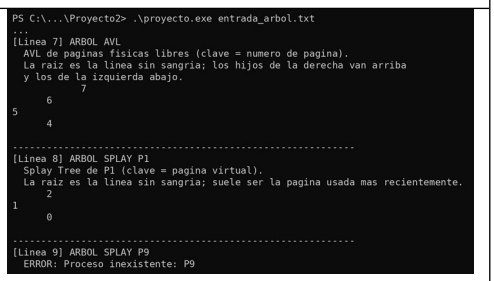
Programa	Qué verifica	Comprobaciones
pruebaMemoria	Los 12 casos mínimos del enunciado, uno por función	68
pruebaParser	Archivo válido, comentarios, BOM y \r, errores de formato, configuración inválida o incompleta, SALIR, modo consola, ARBOL y EXPORTAR	42

Caso manual. Los casos CP-01 a CP-15 son líneas de `entrada_casos.txt`; CP-16 a CP-19 usan los otros archivos de entrada y el modo consola; CP-20 y CP-21 corresponden a los extras (sección 16). La columna Resultado muestra la salida real del programa para cada caso.

Caso	Pasos	Resultado esperado	Resultado
CP-01	Crear un proceso válido: `NEW_PROCESS P1 10240`	Se crea con 10 páginas (físicas 0-9) y caché local de 2	
CP-02	Crear un proceso sin memoria suficiente: `NEW_PROCESS P4 60000` con 48 páginas libres	Se rechaza: necesita 59 páginas	
CP-03	Crear un proceso repetido: `NEW_PROCESS P1 1024`	Se rechaza como proceso duplicado	
CP-04	Acceso exitoso: `ACCESS P1 2500` y luego `ACCESS P1 2600`	Página virtual 2, desplazamiento 452; el primero es MISS y el segundo HIT local	

Caso	Pasos	Resultado esperado	Resultado
CP-05	Acceso a dirección inválida: `ACCESS P1 -1`, `ACCESS P3 5000` y `ACCESS P9 0`	Dirección inválida dos veces y Proceso inexistente	
CP-06	Asignación adicional: `ALLOCATE P2 8192` y `ALLOCATE P3 100000`	P2 pasa a 12 páginas y su caché local a 2; la segunda es Memoria insuficiente	
CP-07	Liberación parcial: `FREE P1 2048 2048` y luego `ACCESS P1 2500`	Se liberan las páginas 2 y 3; el acceso da Acceso a página no válida	
CP-08	Liberación inválida: `FREE P1 2048 1024` y `FREE P3 0 99999`	Se rechazan y no se libera nada	
CP-09	Llenado del caché local: después de `ACCESS P1 0`, ejecutar `ACCESS P1 5000`	El caché local de P1 (2) está lleno y LRU expulsa P1:2	
CP-10	Hit en caché global: `ACCESS P1 2500` después de CP-09	MISS local y HIT global: P1:2 salió del caché local pero sigue en el global	
CP-11	Llenado del caché global: `ACCESS P2 0`	El caché global queda lleno (4/4)	

Caso	Pasos	Resultado esperado	Resultado
CP-12	Reemplazo por LRU: `ACCESS P3 0` con el caché global lleno	LRU expulsa P1:0, la entrada global menos reciente	
CP-13	Múltiples procesos activos: `ACCESS P2 1024` con P1, P2 y P3 activos	Cada proceso tiene su caché local; el global mezcla entradas de los tres y expulsa P1:4	
CP-14	Errores de formato: `MOVE P1 0`, `ACCESS P1 abc` y `NEW_PROCESS P5`	Operación desconocida y dos veces Archivo de entrada incorrecto	
CP-15	Finalización de proceso: `END_PROCESS P1` dos veces	Sus 8 páginas vuelven al AVL y salen del caché global; la segunda vez es Proceso inexistente	
CP-16	Configuración inválida: `proyecto.exe entrada_error_config.txt`	Tamaño de página inválido; la simulación se detiene	
CP-17	Modo consola: opción 2, escribir 65536, 1000 y 8; luego 65536, 1024 y 4	La primera configuración se rechaza y se vuelve a pedir	

Caso	Pasos	Resultado esperado	Resultado
CP-18	Modo consola: `NEW_PROCESS P1 4096`, `ACCESS P1 2500` y `ACCESS P1 2600`	El mismo resultado que en modo archivo; el segundo acceso es HIT local	
CP-19	Ejemplo del enunciado: `proyecto.exe entrada_prueba.txt`	9 operaciones, 0 errores; al final las 64 páginas quedan libres	
CP-20	Dibujo de árboles con 8 páginas físicas: P1 con 3 páginas y un `ACCESS P1 1024`; luego `ARBOL AVL`, `ARBOL SPLAY P1` y `ARBOL SPLAY P9`	El AVL muestra las páginas libres 4 a 7; el Splay de P1 tiene la página 1 en la raíz; P9 da Proceso inexistente	
CP-21	Exportación: `EXPORTAR metricas.csv` y luego mostrar el archivo	Un CSV con las métricas generales y una fila por proceso activo	

19. Conclusiones

El simulador cumple todo lo que pide el enunciado, y cada parte está probada por separado y en conjunto. El AVL resultó la estructura adecuada para las páginas libres: aunque las páginas se insertan en orden, mantiene una altura logarítmica y entrega la página más baja en O(log N). El Splay Tree aprovecha la localidad de referencia de la tabla de páginas, porque las páginas usadas seguido quedan en la raíz. El caché LRU con lista y tabla hash logra O(1) por operación.

Lo más valioso del proyecto fue ver cómo estructuras distintas se complementan: cada una resuelve bien una parte del problema y el MemoryManager las coordina. También quedó claro el valor de probar cada módulo antes de integrarlo: el AVL, el Splay Tree y el caché se validaron por separado, y al juntarlos el simulador funcionó con muy pocos ajustes.